



2-Layer Approval Mechanism

Plan Review & Final Implementation Decisions

Workflow Scheduler | Engineering Review Response

Item	Review Position
Overall concept	Approved. A two-layer approval option is appropriate for governed schedules.
Current plan as written	Requires a few changes before implementation.
Final recommendation	Proceed after adopting the mandatory two-layer progression and explicit approval-assignment schema described in this document.
Implementation status	GREEN after the decisions in Sections 3-8 are incorporated.

1. Review Summary

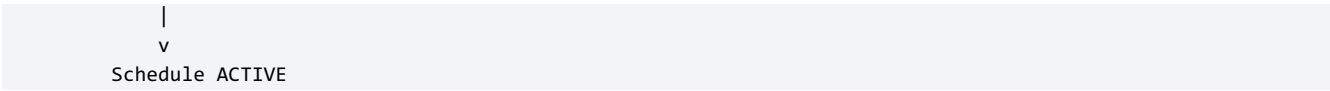
The proposed workflow is directionally correct: the schedule creator can enable a second approval layer, Layer 1 is assigned to an individual reviewer, and Layer 2 is assigned to a Governance Board / approval group. The schedule must remain pending until the required approval path is completed.

Two open questions in the plan require explicit decisions before development: whether Layer 1 can bypass Layer 2, and whether the Layer 1 assignee should be stored in metadata or modeled as a first-class approval assignment.

Final position: When 2-Layer Approval is selected, both layers must be mandatory. Layer 1 approval should automatically advance the request to Layer 2. The approval assignee and approval layer should be explicit schema fields rather than primary routing information stored only in metadata_json.

2. Final Approval Workflow





3. Decision 1 - Two Layers Must Be Mandatory

If the creator explicitly selects Require 2-Layer Approval, Layer 1 must not be able to activate the schedule directly. Allowing Layer 1 to approve and bypass the Governance Board would make the second layer optional and weaken the governance control.

Configuration	Layer 1 APPROVE	Final result
Single-layer approval	Completes required approval	Schedule becomes ACTIVE
Two-layer approval	Completes Layer 1 and creates Layer 2 request	Schedule remains PENDING_APPROVAL
Two-layer approval + Layer 2 APPROVE	Completes final mandatory approval	Schedule becomes ACTIVE

```
if require_2_layer_approval is False:
    layer_1 APPROVED -> schedule ACTIVE

if require_2_layer_approval is True:
    layer_1 APPROVED -> create layer_2 approval
                        -> schedule remains PENDING_APPROVAL

    layer_2 APPROVED -> schedule ACTIVE
```

4. Decision 2 - Do Not Use 'Escalate' as the Normal Layer-2 Transition

The normal two-layer progression should be Approve -> Next Approval Layer. 'Escalate' should be retained, if required, as a separate exceptional governance action rather than the standard mechanism for reaching Layer 2.

- Layer 1 primary action: Approve.
- On approval, the backend automatically completes Layer 1 and creates the Layer 2 approval request.
- The UI should explain: 'Approval will be forwarded to the Governance Board.'
- Escalate can later be used for exceptional escalation to a higher authority, risk committee, or special review path.

5. Decision 3 - Add Explicit Approval Assignment Fields

The current plan suggests using metadata_json.reviewer_user_id to avoid a database migration. This is not recommended for a core governance workflow. Approval assignment is transactional, queryable and auditable data and should be modeled explicitly.

Field	Recommendation / Purpose
assigned_user_id	Nullable FK to user. Used for Layer 1 individual assignment.
approval_group_id	Nullable FK to approval group. Used for Layer 2 group assignment.
approval_layer	Integer / small integer. 1 for reviewer, 2 for Governance Board.
approval_type	USER or GROUP.
parent_approval_id	Optional reference to preceding approval record.
approval_cycle_id	Groups all approval records belonging to one submission/resubmission cycle.
status	PENDING, APPROVED, REJECTED, CHANGES_REQUESTED, CANCELLED.
decided_by	Actual user who made the decision.
decided_at	Decision timestamp.
decision_comment	Approval/rejection/change-request rationale.
metadata_json	Keep for supplementary extensible context, not primary routing.

This small schema migration will materially simplify My Approvals queries, indexes, RBAC, notifications, SLA tracking, delegation, reporting, audit history and future multi-stage approval support.

6. Schedule Status vs Approval Status

Keep the schedule lifecycle simple. Layer-specific state should live in approval records rather than creating multiple schedule status values.

Schedule status	Meaning
DRAFT	Editable / not currently in an active approval submission.
PENDING_APPROVAL	At least one mandatory approval remains outstanding.
ACTIVE	All mandatory approvals completed and schedule activated.

The UI can derive friendly stage labels from the active approval record, for example 'Awaiting Reviewer Approval' or 'Awaiting Governance Board Approval'.

7. Approval Record Example

```
Approval Cycle AC-001

Schedule SCH-001
Status = PENDING_APPROVAL

Approval #1
  Layer = 1
  Type = USER
  Assigned User = User A
  Status = PENDING

After User A approves:

Approval #1
  Layer = 1
  Status = APPROVED
  Decided By = User A

Approval #2
  Layer = 2
  Type = GROUP
  Approval Group = Governance Board
  Status = PENDING
  Parent Approval = Approval #1

Schedule remains PENDING_APPROVAL

After an authorized Governance Board member approves:

Approval #2
  Layer = 2
  Status = APPROVED
  Decided By = User B

Schedule -> ACTIVE
```

8. Layer 2 Group Decision Rule

For the current implementation, one authorized active member of the assigned approval group should be able to make the Layer 2 decision on behalf of the group. Store both the assigned group and the actual user who decided.

- approval_group_id = Governance Board
- decided_by = actual authenticated group member
- Backend verifies active group membership at decision time.
- Future versions can extend this to quorum, N-of-M, unanimous or sequential group approval without changing the basic approval-chain model.

9. Reject, Request Changes and Resubmission

Action	Recommended behavior
Reject	Mark the current approval record/cycle REJECTED. Return schedule for rework according to current scheduler lifecycle.
Request Changes	Return schedule to DRAFT with mandatory feedback. Preserve the completed approval history.
Resubmit	Create a NEW approval_cycle_id and new Layer 1 approval. Do not reuse or overwrite the old approval records.

```
Cycle 1
  Layer 1 APPROVED
  Layer 2 REJECTED

Schedule edited

Cycle 2
  Layer 1 PENDING

Previous cycle remains immutable for governance/audit history.
```

10. Backend Implementation Guidance

10.1 submit_for_approval()

```
validate schedule and governance configuration
create approval_cycle_id

if require_2_layer_approval:
    require reviewer_user_id
    require approval_group_id

create Layer 1 approval:
    approval_type = USER
    assigned_user_id = reviewer_user_id
    approval_layer = 1
    status = PENDING

schedule.status = PENDING_APPROVAL
notify Layer 1 reviewer
```

10.2 decide_approval(APPROVED)

```
verify:
- approval is current and PENDING
- authenticated user is authorized assignee/group member
- approval cycle is current
```

```
- schedule has not materially changed

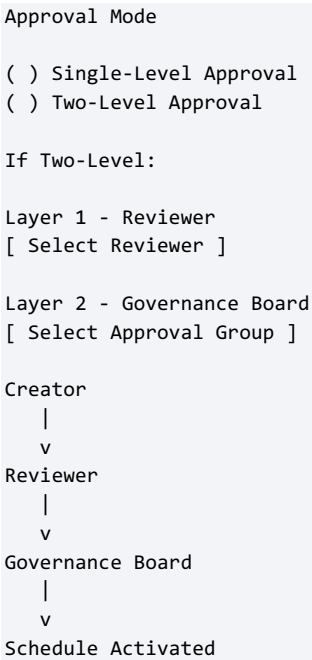
complete current approval

if current.approval_layer == 1 and schedule.require_2_layer_approval:
    create Layer 2 approval:
        approval_type = GROUP
        approval_group_id = schedule.approval_group_id
        approval_layer = 2
        parent_approval_id = current.id
        status = PENDING

    schedule remains PENDING_APPROVAL
    notify Layer 2 group
else:
    verify no mandatory approval remains
    schedule.status = ACTIVE
```

11. Frontend Recommendations

Instead of only a checkbox, present the approval configuration as an explicit mode so the governance path is immediately understandable.



Approval screen	Recommended presentation
Layer 1	Approval Stage: 1 of 2. Primary action: Approve. Helper text: 'Approval will be forwarded to the Governance Board.'
Layer 2	Approval Stage: 2 of 2 - Final Approval. Primary action: Approve & Activate.
Queue	My Approvals uses assigned_user_id; Group Approvals uses active approval_group_id membership.
History	Show Layer, assigned authority, actual decision maker, decision, timestamp and comments.

12. Governance Events / Audit

The approval flow should produce an auditable chain. Event names may be aligned with the existing GuardianIQ event catalogue before implementation.

Event	Trigger
SCHEDULE_APPROVAL_SUBMITTED	Schedule submitted into an approval cycle.
LAYER_1_APPROVAL_REQUESTED	Layer 1 request created.
LAYER_1_APPROVAL_GRANTED	Reviewer approves.
LAYER_1_APPROVAL_REJECTED	Reviewer rejects.
LAYER_2_APPROVAL_REQUESTED	Mandatory Layer 2 request created.
LAYER_2_APPROVAL_GRANTED	Governance Board member approves.
LAYER_2_APPROVAL_REJECTED	Governance Board member rejects.
SCHEDULE_CHANGES_REQUESTED	Reviewer/group requests changes.
SCHEDULE_ACTIVATED	All mandatory approvals completed.

All records/events should carry schedule_id, approval_cycle_id and correlation_id so the full approval journey can be reconstructed.

13. Required Verification Scenarios

Test	Expected result
2-layer schedule submitted	Layer 1 USER approval created; schedule PENDING_APPROVAL.
Layer 1 approves	Layer 1 APPROVED; Layer 2 GROUP approval created; schedule still PENDING_APPROVAL.
Layer 1 attempts to activate directly	Backend prevents bypass.
Unauthorized user decides Layer 1	Rejected by backend.
Non-member decides Layer 2	Rejected by backend.
Layer 2 member approves	Schedule becomes ACTIVE.
Layer 2 rejects	Approval cycle records REJECTED; schedule returned for rework per lifecycle.
Request Changes	Feedback retained; schedule DRAFT; old approval history preserved.
Resubmission	New approval_cycle_id and fresh Layer 1 request.
Old approval link used after resubmission	Rejected as stale/non-current.
Layer 1 assignment query	Appears in assigned user's My Approvals.
Layer 2 assignment query	Appears to authorized group members.
Audit history	Both assignment authority and actual decision maker are visible.

14. Final Green Flag

Decision: GREEN FLAG TO PROCEED, provided the implementation incorporates the mandatory two-layer progression, explicit approval-assignment/layer schema, automatic Layer 1 APPROVE -> Layer 2 transition, and preserved approval-cycle history described above.

The proposed concept should therefore proceed with the following final engineering decisions:

- Two-layer means both layers are mandatory.
- Layer 1 APPROVE automatically advances to Layer 2; it never activates a two-layer schedule.
- Escalate is not the standard Layer 1 -> Layer 2 transition.
- Use assigned_user_id and explicit approval-layer fields; do not rely on metadata_json for core routing.
- Layer 2 can initially be decided by any one authorized active member of the assigned approval group.
- Approval history is immutable across edits/resubmissions; every resubmission creates a new approval cycle.
- Schedule activation occurs only after the backend verifies that no mandatory approval remains.